Projektowanie aplikacji webowych opartych o mikrousługi

Locker Service — rozwiązanie referencyjne LAB1

To jest działające rozwiązanie przykładu wspólnego do analizy podczas LAB1. Pokazuje technikę, regułę biznesową i testy. Własny projekt zbuduj ze startera, używając własnej domeny. Wymagania zaliczenia określa LAB1.

Problem i granice

Kurier potrzebuje sprawdzić dostępność skrytki i zmienić jej stan. `Locker Service` jest właścicielem danych paczkomatów i skrytek. Nie prowadzi historii przesyłek, płatności, kont ani wysyłki powiadomień.

Usługa	Odpowiedzialność	Własne dane	Operacje	Potrzebne informacje
Locker Service — zaimplementowana	Dostępność paczkomatów i skrytek	Paczkomaty, rozmiary i statusy skrytek	Odczyt, dodanie, zmiana statusu	Na LAB1 nie wymaga drugiej usługi
Parcel Service — planowana	Obsługa paczek	Paczki i ich statusy	Umieszczenie paczki, odczyt jej statusu	Dostępność skrytki i potwierdzenie zmiany jej stanu

Pojęcia domenowe: **paczkomat** to lokalizacja zawierająca skrytki; **skrytka** to miejsce na paczkę; **rezerwacja** oznacza czasowe przeznaczenie skrytki dla operacji; **zajętość** oznacza stan po umieszczeniu paczki. Skrytki pozostają razem z paczkomatami, bo tworzą jeden obszar zarządzania dostępnością.

Przyszły kontrakt: po wcześniejszej rezerwacji `Parcel Service` wywoła `PATCH /compartments/{id}/status` z `{"status":"occupied"}`. `Locker Service` zwróci `200` i obiekt skrytki albo `404` / `409` / `422`. Przy niedostępności odbiorcy operacja umieszczenia paczki pozostaje niepotwierdzona; nie wolno zgłosić sukcesu ani opublikować `ParcelPlaced`. Sposób rozstrzygania nieznanego wyniku po timeout i niezawodną publikację zdarzeń projektujemy później. Komunikacja między usługami nie jest zaimplementowana na LAB1.

Model i reguła biznesowa

`Locker` zawiera `id`, `name`, `address`, `city`, `status` (`online` / `offline`). `Compartment` zawiera `id`, `locker_id`, `size` (`S` / `M` / `L`) i `status`. Identyfikatory podaje klient. Identyfikator skrytki jest unikalny w całej usłudze. Przy uruchomieniu powstaje paczkomat `KRK-1024` oraz skrytki `KRK-1024-L1` i `KRK-1024-M1` w stanie `free`.

Reguła biznesowa: można przechodzić tylko `free → reserved → occupied → free` lub `free ↔ out_of_service`. Ponowne ustawienie aktualnego statusu jest dozwolone. To uproszczony model dydaktyczny, bez anulowania i wygasania rezerwacji. Przykładowo `free → reserved` jest poprawne, a `reserved → out_of_service` zwraca `409` bez zmiany danych.

Kontrakt API

Metoda i ścieżka	Sukces	Błędy
`GET /health`	200, status i nazwa usługi	—
`GET /lockers`	200, lista paczkomatów	—
`GET /lockers/{locker_id}`	200, obiekt paczkomatu	404
`POST /lockers`	201, utworzony paczkomat	409 duplikat, 422 dane
`GET /lockers/{locker_id}/compartments`	200, lista skrytek (może być pusta)	404 brak paczkomatu
`POST /lockers/{locker_id}/compartments`	201, skrytka w stanie free	404, 409 duplikat, 422 dane
`PATCH /compartments/{compartment_id}/status`	200, skrytka po zmianie	404, 409 przejście, 422 dane

Status `404` oznacza brak zasobu, `409` konflikt z aktualnym stanem, a `422` błędne dane wejściowe. Błędy `404` / `409` mają `{"detail":"opis"}`. Błędy walidacji `422` mają listę obiektów w `detail`, wskazujących m.in. pole (`loc`) i przyczynę (`msg`). Ograniczenia pól i przykłady modeli są dostępne w `/docs`.

Uruchomienie

Wymagany Python 3.12 lub nowszy. Polecenia wykonaj w PowerShellu w katalogu `locker-service`, zawierającym `requirements.txt`:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
```

Dokumentacja API:

- http://localhost:8000/docs
- http://localhost:8000/redoc

Testy

```
.\.venv\Scripts\python.exe -m pytest -q
```

Nie trzeba uruchamiać Uvicorna do testów `TestClient`. Fixture tworzy świeżą aplikację i magazyn dla każdego testu. Testy sprawdzają zapis i odczyt, dozwolone przejścia, błędne dane, duplikat, niedozwolone przejście, brak zasobu i niezależność stanu aplikacji. Po odrzuceniu zapisu porównują dane z poprzednim stanem. Test nowej aplikacji nie zastępuje ręcznego eksperymentu z rzeczywistym restartem procesu.

Przykładowe żądania

```
Invoke-RestMethod http://localhost:8000/health
Invoke-RestMethod http://localhost:8000/lockers
Invoke-RestMethod http://localhost:8000/lockers/KRK-1024/compartments
```

Dodanie paczkomatu:

```
$lockerBody = @{ id = 'WAW-0001'; name = 'Warszawa Centrum'; address = 'ul. Testowa 1'; city = 'Warszawa' } | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri http://localhost:8000/lockers -ContentType 'application/json' -Body $lockerBody
Invoke-RestMethod http://localhost:8000/lockers/WAW-0001
```

Obie odpowiedzi zawierają dane paczkomatu oraz `"status":"online"`. Statusy HTTP to odpowiednio `201` i `200`; można je obejrzeć w `/docs` (PowerShell powyżej pokazuje treść). Powtórny `POST` zwraca `409` i nie zmienia istniejącego paczkomatu.

Zmiana statusu początkowo wolnej skrytki i osobny odczyt:

```
Invoke-RestMethod -Method Patch -Uri http://localhost:8000/compartments/KRK-1024-L1/status -ContentType 'application/json' -Body
Invoke-RestMethod http://localhost:8000/lockers/KRK-1024/compartments
```

Odpowiedź `PATCH` (`200`):

```
{"id":"KRK-1024-L1","size":"L","locker_id":"KRK-1024","status":"reserved"}
```

Następnie w `/docs` spróbuj dla tej skrytki `{"status":"out_of_service"}`. Oczekuj `409` z `{"detail":"Cannot change compartment status from 'reserved' to 'out_of_service'"}`. Ponowny `GET` powinien nadal pokazać `reserved`.

Powtarzalny scenariusz ręczny

Zacznij od restartu Uvicorna. Podczas scenariusza nie zapisuj zmian w kodzie, żeby `--reload` nie resetował danych. Wszystkie operacje wykonasz w `/docs`; zawsze sprawdzaj status HTTP i treść odpowiedzi.

Krok	Żądanie / działanie	Oczekiwany wynik
1	`GET /health`	200, `{"status":"ok","service":"locker-service"}`
2	`GET /lockers` i `GET /lockers/KRK-1024`	200, lista i szczegóły paczkomatu KRK-1024
3	`POST /lockers` z JSON-em paczkomatu WAW-0001 z przykładu powyżej	201, obiekt z tym id
4	`GET /lockers/WAW-0001`	200, dane zapisane w kroku 3
5	Powtórz krok 3 ze zmienionym `city`, potem `GET /lockers/WAW-0001`	409, a następnie 200 z niezmienionymi danymi
6	`GET /lockers/UNKNOWN`	404, `detail` opisuje brak paczkomatu
7	`PATCH /compartments/KRK-1024-L1/status`, JSON `{"status":"broken-value"}`; potem `GET /lockers/KRK-1024/compartments`	422, `detail` wskazuje status; skrytka nadal free
8	Ta sama ścieżka PATCH, JSON `{"status":"reserved"}`; potem GET skrytek	200, w obu odpowiedziach skrytka L1 ma reserved
9	Ta sama ścieżka PATCH, JSON `{"status":"out_of_service"}`; potem GET skrytek	409; skrytka L1 nadal reserved
10	Zatrzymaj serwer przez Ctrl+C i uruchom ponownie	Nowy magazyn z danymi początkowymi
11	`GET /lockers/WAW-0001` i `GET /lockers/KRK-1024/compartments`	404 dla WAW-0001, skrytki KRK-1024 ponownie free

Zapisz własny wynik rzeczywisty dla każdego kroku. Restart resetuje cały scenariusz. Dane żyją w magazynie aplikacji między żądaniami, ale nie są zapisywane poza pamięcią procesu.

Porównanie architektur

```
Monolit:            klient → [jeden proces: paczki + skrytki + powiadomienia] → wspólne dane
Modularny monolit:  klient → [jeden proces: moduł paczek → moduł skrytek]
                                           wywołanie w kodzie; logiczne granice danych
Mikrousługi:        klient → [Parcel Service, dane paczek] → HTTP → [Locker Service, dane skrytek]
                             osobny proces                         osobny proces
```

W modularnym monolicie moduły i własność danych są wyraźnie rozdzielone, ale wdrożenie obejmuje całą aplikację. `Locker Service` jest samodzielną aplikacją z własnym magazynem i kontraktem, więc może stać się częścią systemu mikrousługowego. Sam działający endpoint nie dowodzi jeszcze niezależności całego systemu. Dla małej aplikacji paczkomatów modularny monolit może być prostszy; mikrousługi umożliwiają niezależne wdrażanie, ale wymagają obsługi opóźnień, niedostępności i częściowych błędów komunikacji. Awaria jednej usługi może wpłynąć na zależne operacje.

Dokumentacja a implementacja

Fragment dokumentacji	Zastosowanie	Weryfikacja
FastAPI: Request Body	Modele żądań Pydantic i walidacja statusu	Test błędnego statusu oczekuje 422, wskazania pola i niezmienionego stanu
FastAPI: Handling Errors	Tłumaczenie NotFoundError i ConflictError na HTTPException	Testy braku zasobu, duplikatu i niedozwolonego przejścia
FastAPI: Testing	TestClient i asercje odpowiedzi	Testy API bez uruchamiania Uvicorna

W swoim README opisz własne wykorzystanie dokumentacji, autorów oraz użycie AI. Ten przykład nie jest gotowym sprawozdaniem studenta.

Ograniczenia przykładu

- dane są przechowywane tylko w pamięci procesu,
- brak autoryzacji,
- brak bazy danych,
- brak RabbitMQ, Eureki, Gatewaya i Dockera,
- uproszczone przejścia statusów, bez anulowania, wygasania i identyfikatora rezerwacji,
- status paczkomatu online/offline jest informacyjny; przykład nie modeluje urządzenia ani fizycznego umieszczenia paczki,
- magazyn nie zapewnia transakcji ani ochrony przed równoczesnymi zapisami; ćwiczymy sekwencyjne żądania w jednej instancji,
- każda instancja ma osobne dane, a restart i `--reload` przywracają dane początkowe.